OWASP Top 10 Vulnerabilities — Research

Vortextech Cybersec Week 1

Introduction

The OWASP Top 10 is a list, published by the Open Worldwide Application Security Project, of the ten most critical security risks facing web applications. It is updated every few years and is treated as an industry-standard starting point for secure development. The most recent edition is the 2025 version. Below I document five of these risks in my own words: what each one is, how an attacker could exploit it, a real-world example, and a practical way to prevent it.

The current 2025 Top 10, for reference:

1. Broken Access Control
2. Security Misconfiguration
3. Software Supply Chain Failures
4. Cryptographic Failures
5. Injection
6. Insecure Design
7. Authentication Failures
8. Software or Data Integrity Failures
9. Security Logging & Alerting Failures
10. Mishandling of Exceptional Conditions

1. Broken Access Control (A01:2025)

What it is: Access control is the set of rules that decide who is allowed to do what in an app; like which pages you can view or which records you can edit. "Broken" access control means those rules are not enforced properly, so a normal user can reach things they should not: other people's data, admin-only features, or actions meant for someone else. It is ranked #1 because it is both extremely common and high-impact.

How an attacker exploits it: A very common trick is changing an ID in a URL or request. Imagine viewing your invoice at example.com/invoice?id=123. If you change it to id=124 and

the app shows you someone else's invoice, that is a broken access control flaw (often called an Insecure Direct Object Reference, or IDOR). Attackers simply increment or guess IDs, or send requests to admin URLs directly, to harvest data or perform privileged actions.

Real-world example: In 2019, the real estate company First American Financial was found to be exposing around 885 million sensitive records (bank details, Social Security numbers, mortgage documents) through a website flaw. Anyone with a valid document link could simply change the number in the URL to view other customers' documents; a textbook IDOR / broken access control failure.

How to prevent it: Enforce access checks on the server side for every request, not in the browser. Deny by default, and verify that the logged-in user actually owns or is authorized for the specific record they are requesting, do not trust an ID coming from the client as proof of permission.

2. Injection (A05:2025)

What it is: Injection happens when an app takes input from a user and mixes it directly into a command or query without separating "data" from "instructions." The classic case is SQL injection, where user input becomes part of a database query. If the app is not careful, an attacker can type in text that the database reads as commands instead of plain data.

How an attacker exploits it: Say a login form builds a query like SELECT * FROM users WHERE name = '[input]'. If an attacker enters ' OR '1'='1 as the username, the query becomes always-true and may log them in without a password or dump the whole users table. Similar attacks exist for operating-system commands and other interpreters.

Real-world example: In 2015, UK telecom TalkTalk suffered a breach exposing the personal data of about 157,000 customers after attackers used SQL injection against a vulnerable web page. The UK Information Commissioner's Office fined TalkTalk £400,000, noting the attack exploited a well-known and preventable weakness.

How to prevent it: Use parameterized queries (prepared statements) so user input is always treated as data, never as executable code. Validating and escaping input helps too, but parameterization is the reliable fix.

3. Cryptographic Failures (A04:2025)

What it is: This is about failing to protect sensitive data properly, either not encrypting it at all, or using weak/outdated encryption. It covers things like storing passwords in plain text, using broken algorithms, or sending data over unencrypted connections. (In older lists this was called "Sensitive Data Exposure.")

How an attacker exploits it: If a database is stolen and passwords are stored in plain text or with weak, unsalted hashing, the attacker instantly knows everyone's passwords, and can reuse them on other sites. If traffic is not encrypted (no HTTPS), an attacker on the same network can read data in transit, including login credentials.

Real-world example: The 2013 Adobe breach exposed around 153 million accounts. Adobe had encrypted passwords poorly, using a reversible cipher in a weak mode without proper salting, and stored password hints in plain text. This let researchers and attackers deduce huge numbers of the actual passwords, making the leak far worse than it needed to be.

How to prevent it: Never store passwords reversibly. Hash them with a strong, slow, salted algorithm designed for passwords (e.g. bcrypt, scrypt, or Argon2), and encrypt sensitive data both in transit (HTTPS/TLS) and at rest.

4. Security Misconfiguration (A02:2025)

What it is: The software might be fine, but it is set up insecurely. This includes leaving default usernames/passwords in place, enabling unnecessary features, exposing detailed error messages, or leaving cloud storage open to the public. It jumped to #2 in 2025 because so many systems are misconfigured.

How an attacker exploits it: Attackers scan the internet for common mistakes, a default admin/admin login, an exposed admin dashboard, a database with no password, or a cloud storage bucket set to "public." When they find one, no clever hacking is needed; the door is simply left open.

Real-world example: In 2021, a misconfiguration in Microsoft Power Apps default settings caused around 38 million records across many organizations' portals to be publicly accessible, including names, addresses, and COVID-19 vaccination data. The software worked as designed; the configuration exposed data that should have been private.

How to prevent it: Harden configurations: change or remove default credentials, disable features you do not use, keep error messages generic, and use a repeatable, reviewed setup process and regular audits so nothing is accidentally left open.

5. Authentication Failures (A07:2025)

What it is: This covers weaknesses in how an app confirms who you are, the login process. Examples include allowing weak passwords, not limiting login attempts, weak or exposed session tokens, and not defending against attackers reusing passwords leaked from other sites.

How an attacker exploits it: A common attack is credential stuffing: attackers take username/password pairs leaked from one breached site and automatically try them on many other sites, betting that people reuse passwords. If there is no rate-limiting or multi-factor authentication, a large share of those attempts succeed.

Real-world example: In 2023, genetics company 23andMe disclosed that attackers used credential stuffing, reused passwords from earlier breaches to break into accounts, ultimately affecting the data of roughly 6.9 million people through connected "relatives" features. No flaw in 23andMe's own encryption was needed; weak/reused authentication was enough.

How to prevent it: Require multi-factor authentication (MFA), enforce strong password policies, limit and slow down repeated login attempts, and check new passwords against known breached password lists.

Personal Reflection

The vulnerability that surprised me most was Cryptographic Failures. What struck me is that it has nothing to do with social engineering or tricking a person, yet the 2013 Adobe breach still exposed around 153 million accounts. It really shows how a small technical oversight, like storing passwords with weak, reversible encryption instead of proper hashing, can have a huge impact. It made me realize that getting the small, behind the scenes details right can matter just as much as defending against the obvious attacks.